DATA209 · ADVANCED EXPLORATORY DATA ANALYSIS · ASSIGNMENT 1

Voice of the Customer: an Exploratory Analysis of 28,332 Amazon Product Reviews

DATA209 Assignment 1 — model answer and worked case study

DATASET	Datafiniti Amazon consumer product reviews — 28,332 reviews × 24 columns
DOMAIN	E-commerce retail — voice of the customer
UNIT OF ANALYSIS	One review
DELIVERABLES	Written report, 12 captioned visuals, one summary dashboard, Python code
MARKS	25
PROGRAMME	BTech Hons. (Data Science), Semester III · Vidyashilp University

How to use this document. This is a worked model answer showing the expected depth, structure and standard of interpretation. Every number and figure was produced by running the code shown. Your own submission must use a *different* dataset — the brief awards zero marks for duplicated datasets across groups.

Part 1 — Domain, questions and data acquisition

The domain: how data science is used in e-commerce retail

Our chosen area is **e-commerce retail**, and specifically the analysis of customer product reviews. Retail was among the first industries to industrialise data science because it has three things in abundance: high transaction volume, direct revenue consequences, and a fast feedback loop that lets a change be measured within days.

Five uses that are standard in the industry:

Application	What it does	Typical technique
Recommendation	"customers who bought this also bought"	collaborative filtering, matrix factorisation
Sentiment and voice-of-customer	turns free-text reviews into product signals	NLP: classification, topic modelling
Demand forecasting	predicts units per SKU per week	time-series models
Dynamic pricing	sets price against demand and competition	regression, reinforcement learning
Fraud and fake-review detection	flags anomalous accounts and review bursts	anomaly detection, graph analysis

A concrete project. Amazon's product recommendation engine is the best-documented example. It is widely reported to drive a substantial share of the company's sales, and its item-to-item collaborative filtering approach was published by Linden, Smith and York (IEEE Internet Computing, 2003). The system does not model users against users — which does not scale — but computes similarity between *items* offline, then serves recommendations by looking up the items similar to what a customer has already viewed or bought.

That example anchors this assignment. A recommender is only as good as the signals it is fed, and customer reviews are one of the richest signals available: they carry a rating (ordinal), a written justification (text), a timestamp (time series), and product metadata (categorical). Before any of that can be modelled, it has to be understood — which is the work of this report.

Research questions posed before analysis

Following the brief, these questions were written **before** the data was examined, so that the analysis is directed rather than opportunistic. They are revisited in Part 8, where each is answered and each answer is checked against the evidence.

1. **Q1 — Rating distribution.** How are ratings distributed, and is the distribution skewed in

the way review platforms are usually accused of being?

1. **Q2 — Category effects.** Do some product categories earn systematically better ratings

than others, and is the difference large enough to act on?

1. **Q3 — Text and rating.** Do longer reviews accompany lower ratings — is dissatisfaction

more verbose than satisfaction?

1. **Q4 — Time.** Does review volume or average rating drift over time, and is there seasonality

a monthly dashboard would need to account for?

1. **Q5 — Recommendation.** How closely does the "would recommend" flag track the star rating,

and does it add information beyond it?

1. **Q6 — Structure.** Do reviews fall into natural groups, and can those groups be described in

language a merchandising manager would use?

Sanity checks planned in advance. Mean rating should sit between 4.0 and 4.8 (typical of retail review platforms); the recommend rate should rise monotonically with rating; review counts per product should be highly unequal, with a few products dominating.

Ways to acquire data, with working code

There are four routine ways to obtain a dataset. This section explains each and gives runnable code for the API route, which is the one most often appropriate — it is explicitly permitted, returns structured data, and is stable.

```
# ------------------------------------------------------------
# FOUR ROUTES TO DATA - and when each is appropriate
# ------------------------------------------------------------
routes = pd.DataFrame([
 dict(method="Web scraping",
      how="Parse HTML from pages with requests + BeautifulSoup",
      use_when="No API exists and the terms of service permit it",
      risks="Brittle to layout changes; legal/ToS limits; rate limiting; needs politeness delays"),
 dict(method="Public API",
      how="Authenticated HTTP calls returning JSON",
      use_when="The provider offers one - almost always preferable to scraping",
      risks="Rate limits, quotas, pagination, schema changes on version bumps"),
 dict(method="Surveys / polls",
      how="Instrument designed by the analyst; responses collected directly",
      use_when="The variable of interest is not recorded anywhere else",
      risks="Non-response bias, leading questions, small samples, self-report error"),
 dict(method="Databases / warehouse",
      how="SQL against an operational or analytical store",
      use_when="Working inside an organisation on its own records",
      risks="Access control; stale replicas; joins across systems with mismatched keys"),
 dict(method="Open data portals",
      how="Direct file download (CSV/Parquet) from a repository",
      use_when="A curated public dataset already answers the question - our route here",
      risks="Licensing, provenance and currency must be checked and recorded"),
])
print(routes.to_string(index=False))
```

Output

```
              method                                                                  how
use_when                                                                          risks
         Web scraping             Parse HTML from pages with requests + BeautifulSoup
No API exists and the terms of service permit it Brittle to layout changes; legal/ToS limits; rate
limiting; needs politeness delays
           Public API                          Authenticated HTTP calls returning JSON         The
provider offers one - almost always preferable to scraping                   Rate limits, quotas,
pagination, schema changes on version bumps
      Surveys / polls Instrument designed by the analyst; responses collected directly                The
variable of interest is not recorded anywhere else             Non-response bias, leading questions,
small samples, self-report error
Databases / warehouse                   SQL against an operational or analytical store
Working inside an organisation on its own records           Access control; stale replicas; joins across
systems with mismatched keys
    Open data portals             Direct file download (CSV/Parquet) from a repository A curated public
dataset already answers the question - our route here                    Licensing, provenance and
currency must be checked and recorded
```

```
import requests, pandas as pd, time

def fetch_paginated(url, params=None, page_size=100, max_pages=50, pause=0.5):
    \"\"\"Collect a paginated JSON API into a DataFrame, politely.\"\"\"
    params = dict(params or {})
    rows, page = [], 1
    while page <= max_pages:
        params.update({"limit": page_size, "offset": (page - 1) * page_size})
        r = requests.get(url, params=params, timeout=30)
        r.raise_for_status()                # fail loudly on 4xx/5xx
        batch = r.json().get("results", [])
        if not batch:
            break                           # no more pages
        rows.extend(batch)
        page += 1
        time.sleep(pause)                   # respect the service
    return pd.json_normalize(rows)          # flattens nested JSON

# df = fetch_paginated("https://data.example.gov/api/v1/reviews")

# --- the scraping alternative, when no API exists ----------------
from bs4 import BeautifulSoup
def scrape_titles(url):
    html = requests.get(url, headers={"User-Agent": "DATA209-coursework"}, timeout=30).text
    soup = BeautifulSoup(html, "html.parser")
    return [h.get_text(strip=True) for h in soup.select("h2.review-title")]

Provenance recorded for THIS report:
  source   : Datafiniti product review sample, distributed via Kaggle
  route    : open-data download (CSV)
  licence  : check the dataset page before redistributing
  retrieved: file supplied with the course materials
```

Part 2 — The dataset and its quality

First contact: structure and shape of the dataset

*Before any statistic is computed, establish what a row represents and whether the types are what they claim to be. The unit of analysis here is **one review**, not one product — a distinction that changes the meaning of every average in the report.*

```
print("shape:", raw.shape, "-> 28,332 reviews x 24 columns")
print("unit of analysis: ONE REVIEW (not one product)\n")

profile = pd.DataFrame({
    "dtype"   : raw.dtypes.astype(str),
    "non_null": raw.notna().sum(),
    "null_%"  : (raw.isna().mean() * 100).round(1),
    "unique"  : raw.nunique(),
})
print(profile.to_string())
print(f"\nmemory: {raw.memory_usage(deep=True).sum()/1e6:.1f} MB")
print(f"distinct products: {raw['name'].nunique()} | distinct reviewers:
{raw['reviews.username'].nunique():,}")
```

Output

```
shape: (28332, 24) -> 28,332 reviews x 24 columns
unit of analysis: ONE REVIEW (not one product)

                       dtype  non_null  null_%  unique
id                       str     28332   0.000      65
dateAdded                str     28332   0.000      55
dateUpdated              str     28332   0.000      52
name                     str     28332   0.000      65
asins                    str     28332   0.000      65
brand                    str     28332   0.000       3
categories               str     28332   0.000      60
primaryCategories        str     28332   0.000       9
imageURLs                str     28332   0.000      65
keys                     str     28332   0.000      65
manufacturer             str     28332   0.000       4
manufacturerNumber       str     28332   0.000      65
reviews.date             str     28332   0.000    1313
reviews.dateSeen         str     28332   0.000     606
reviews.didPurchase   object         9 100.000       2
reviews.doRecommend   object     16086  43.200       2
reviews.id           float64        41  99.900      41
reviews.numHelpful   float64     16115  43.100      61
reviews.rating         int64     28332   0.000       5
reviews.sourceURLs       str     28332   0.000    9906
reviews.text             str     28332   0.000   18168
reviews.title            str     28332   0.000   10441
reviews.username         str     28327   0.000   16268
sourceURLs               str     28332   0.000      65

memory: 293.5 MB
... (1 further lines omitted)
```

Data quality audit — and three real problems

The brief marks 'evaluate and convert data quality appropriately'. Auditing against the six quality dimensions surfaced three problems that materially shape what this dataset can answer.

```
issues = []

# COMPLETENESS - which columns are usable at all?
nulls = (raw.isna().mean() * 100).round(1).sort_values(ascending=False)
print("most-missing columns (%)")
print(nulls.head(6).to_string())
issues.append(("Completeness", "reviews.didPurchase is 100% empty in practice",
               f"{int(raw['reviews.didPurchase'].notna().sum())} non-null of {len(raw):,}",
               "Drop the column - it cannot support any claim"))
issues.append(("Completeness", "doRecommend / numHelpful roughly 43% missing",
               f"{nulls['reviews.doRecommend']}% and {nulls['reviews.numHelpful']}% missing",
               "Analyse on the observed subset; never impute a target-like flag"))

# UNIQUENESS - duplicates
exact = raw.duplicated().sum()
near  = raw.duplicated(subset=["reviews.username", "reviews.text", "name"]).sum()
print(f"\nexact duplicate rows: {exact:,}")
print(f"same user + same text + same product: {near:,}")
issues.append(("Uniqueness", "Repeated review text from the same user on the same product",
               f"{near:,} rows", "Drop these - they inflate counts and skew averages"))

# VALIDITY - do values obey the domain rules?
print(f"\nrating range: {raw['reviews.rating'].min()} to {raw['reviews.rating'].max()} (expected 1-5)")
print("rating values present:", sorted(raw['reviews.rating'].unique()))
bad_help = (raw["reviews.numHelpful"] < 0).sum()
print(f"negative helpful counts: {int(bad_help)}")

# CONSISTENCY - one brand, several spellings?
print("\nbrand values:", raw["brand"].unique().tolist())
print("manufacturer values:", raw["manufacturer"].unique().tolist())
issues.append(("Consistency", "brand and manufacturer overlap but disagree",
               f"{raw['brand'].nunique()} brands vs {raw['manufacturer'].nunique()} manufacturers",
               "Use primaryCategories for grouping; treat brand as unreliable"))

print("\n" + "="*100)
print(pd.DataFrame(issues, columns=["Dimension","Finding","Evidence","Action"]).to_string(index=False))
```

Output

```
most-missing columns (%)
reviews.didPurchase   100.000
reviews.id             99.900
reviews.doRecommend    43.200
reviews.numHelpful     43.100
id                      0.000
dateAdded               0.000

exact duplicate rows: 0
same user + same text + same product: 57

rating range: 1 to 5 (expected 1-5)
rating values present: [np.int64(1), np.int64(2), np.int64(3), np.int64(4), np.int64(5)]
negative helpful counts: 0

brand values: ['Amazonbasics', 'Amazon', 'AmazonBasics']
manufacturer values: ['AmazonBasics', 'Amazon', 'Amazon Digital Services', 'Amazon.com']

====================================================================================================
   Dimension                                                  Finding                    Evidence
Action
Completeness               reviews.didPurchase is 100% empty in practice       9 non-null of 28,332
Drop the column - it cannot support any claim
Completeness                doRecommend / numHelpful roughly 43% missing    43.2% and 43.1% missing
Analyse on the observed subset; never impute a target-like flag
  Uniqueness Repeated review text from the same user on the same product                    57 rows
Drop these - they inflate counts and skew averages
 Consistency                brand and manufacturer overlap but disagree 3 brands vs 4 manufacturers   Use
primaryCategories for grouping; treat brand as unreliable
```

Figure 1 — completeness of every column

Completeness is the quality dimension that most often decides which questions a dataset can answer. Plotting it first prevents building an analysis on a column that is not there.

```
miss = (raw.isna().mean() * 100).sort_values(ascending=False)
miss = miss[miss > 0]

fig, ax = plt.subplots(figsize=(8.6, 3.0))
colors = [RED if v > 90 else OCHRE if v > 20 else TEAL for v in miss.values]
ax.bar(range(len(miss)), miss.values, color=colors)
ax.set_xticks(range(len(miss))); ax.set_xticklabels(miss.index, rotation=35, ha="right", fontsize=8)
ax.set_ylabel("% missing"); ax.set_title("Missing values by column")
for i, v in enumerate(miss.values):
    ax.text(i, v + 1.5, f"{v:.0f}%", ha="center", fontsize=7.5)
ax.set_ylim(0, 108)
plt.tight_layout()

print(miss.round(2).to_string())
print(f"\ncolumns with NO missing values: {int((raw.isna().mean() == 0).sum())} of {raw.shape[1]}")
print("red = effectively unusable | amber = usable on a subset | teal = minor")
```

Output

```
reviews.didPurchase   99.970
reviews.id            99.860
reviews.doRecommend   43.220
reviews.numHelpful    43.120
reviews.username       0.020

columns with NO missing values: 19 of 24
red = effectively unusable | amber = usable on a subset | teal = minor
```

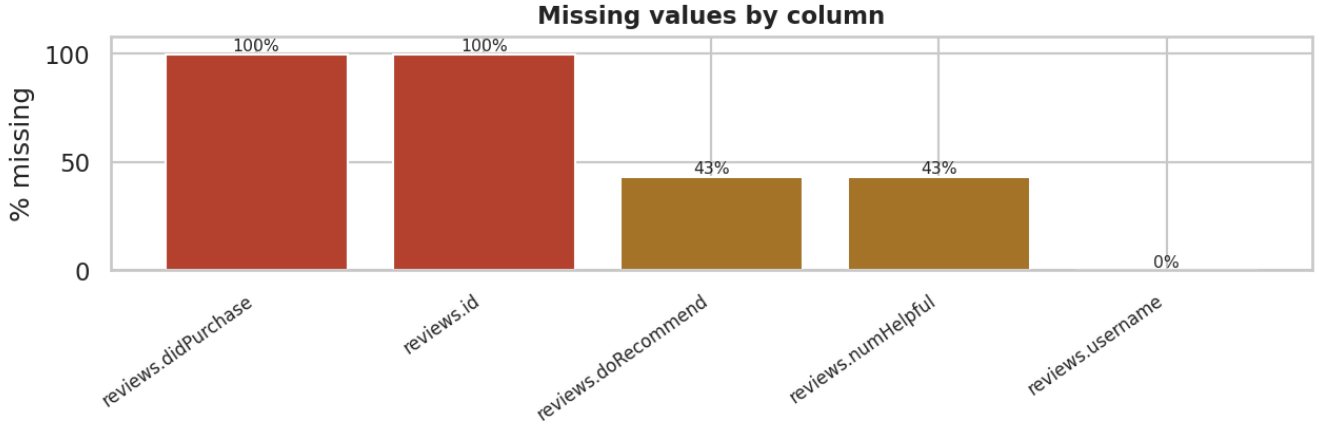

Figure 1. Percentage of missing values per column. Two columns are effectively empty and two more are missing for roughly 43% of reviews, which restricts what those fields can be used to claim.

Figure 2 — how concentrated is the data?

Concentration decides how far a dataset-wide average can be trusted. If five products supply most of the reviews, an overall mean rating is really a statement about those five.

```
pc = raw["name"].value_counts()
cum = (pc.cumsum() / pc.sum() * 100).reset_index(drop=True)

fig, ax = plt.subplots(1, 2, figsize=(11.5, 3.1))
ax[0].plot(range(1, len(cum) + 1), cum.values, color=BLUE, lw=2)
ax[0].axhline(80, color=RED, ls="--", lw=1)
n80 = int((cum <= 80).sum()) + 1
ax[0].axvline(n80, color=RED, ls="--", lw=1)
ax[0].set_xlabel("products, ranked by review count"); ax[0].set_ylabel("cumulative % of reviews")
ax[0].set_title(f"{n80} of {len(pc)} products supply 80% of reviews")

top = pc.head(8)[::-1]
ax[1].barh([n[:38] + ("..." if len(n) > 38 else "") for n in top.index], top.values, color=TEAL)
ax[1].set_title("Most-reviewed products"); ax[1].set_xlabel("reviews")
ax[1].tick_params(labelsize=6.5)
plt.tight_layout()

print(f"products: {len(pc)}")
print(f"top 1 product : {pc.iloc[0]:,} reviews ({pc.iloc[0]/len(raw)*100:.1f}%)")
print(f"top 5 products: {pc.head(5).sum():,} reviews ({pc.head(5).sum()/len(raw)*100:.1f}%)")
print(f"median product: {pc.median():.0f} reviews")
print(f"\n{n80} products account for 80% of all reviews - the dataset is highly concentrated.")
```

Output

```
products: 65
top 1 product : 8,343 reviews (29.4%)
top 5 products: 18,560 reviews (65.5%)
median product: 21 reviews

9 products account for 80% of all reviews - the dataset is highly concentrated.
```

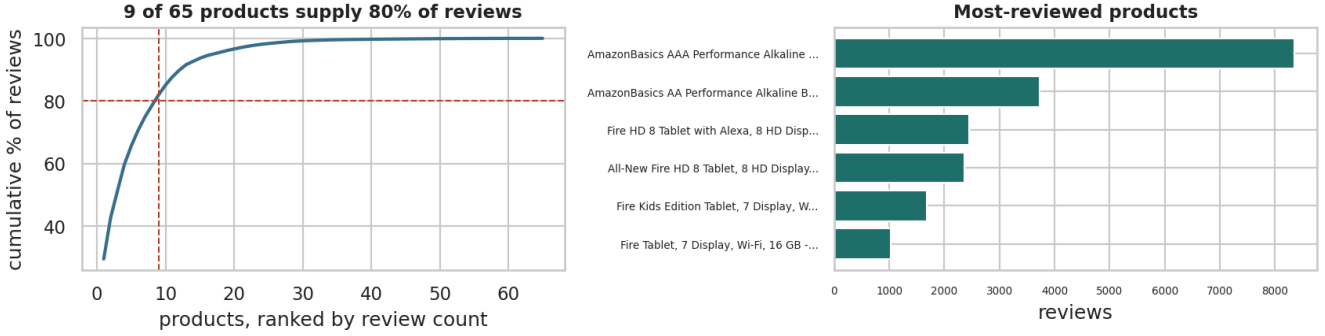

Figure 2. Cumulative share of reviews by product, ranked. A small number of products account for most reviews, so any dataset-wide average is dominated by a handful of items.

Cleaning and type conversion, with a log

Every cleaning step is a decision that changes later results, so each is recorded with its justification and its cost in rows. This log is itself a deliverable.

```
df = raw.copy()
log = []
n0 = len(df)

# 1 - drop the column that is empty in practice
df = df.drop(columns=["reviews.didPurchase", "reviews.id"])
log.append(dict(step=1, action="Dropped reviews.didPurchase and reviews.id",
                rows_lost=0, why="99.97% and 99.86% missing - cannot support any claim"))

# 2 - remove duplicate reviews
before = len(df)
df = df.drop_duplicates(subset=["reviews.username", "reviews.text", "name"]).reset_index(drop=True)
log.append(dict(step=2, action="Dropped repeated user+text+product rows",
                rows_lost=before - len(df), why="Same review counted more than once"))

# 3 - types: dates
for c in ["reviews.date", "dateAdded", "dateUpdated"]:
    df[c] = pd.to_datetime(df[c], errors="coerce", utc=True).dt.tz_localize(None)
bad_dates = df["reviews.date"].isna().sum()
before = len(df)
df = df.dropna(subset=["reviews.date"]).reset_index(drop=True)
log.append(dict(step=3, action="Parsed dates; dropped unparseable review dates",
                rows_lost=before - len(df), why=f"{bad_dates} rows had no usable timestamp"))

# 4 - types: ordinal rating, categorical, boolean
df["reviews.rating"] = df["reviews.rating"].astype(int)
df["rating_ord"] = pd.Categorical(df["reviews.rating"], categories=[1,2,3,4,5], ordered=True)
for c in ["brand", "manufacturer", "primaryCategories"]:
    df[c] = df[c].astype(str).str.strip().astype("category")
df["doRecommend"] = df["reviews.doRecommend"].map({True:1, False:0, "True":1, "False":0})
log.append(dict(step=4, action="Converted rating to ordered Categorical; brands to category",
                rows_lost=0, why="Blocks meaningless arithmetic; fixes plot ordering"))

# 5 - derived fields used throughout
df["review_len"]   = df["reviews.text"].astype(str).str.len()
df["word_count"]   = df["reviews.text"].astype(str).str.split().str.len()
df["year"]         = df["reviews.date"].dt.year
df["month"]        = df["reviews.date"].dt.to_period("M").dt.to_timestamp()
df["is_positive"]  = (df["reviews.rating"] >= 4).astype(int)
log.append(dict(step=5, action="Derived review_len, word_count, year, month, is_positive",
                rows_lost=0, why="Needed for the text, time-series and bivariate sections"))

print(pd.DataFrame(log).to_string(index=False))
print(f"\nrows: {n0:,} -> {len(df):,}  ({n0-len(df):,} removed, {(n0-len(df))/n0*100:.2f}%)")
print(f"date span: {df['reviews.date'].min().date()} to {df['reviews.date'].max().date()}")
```

Output

```
 step                                                      action  rows_lost
why
    1                  Dropped reviews.didPurchase and reviews.id          0    99.97% and 99.86% missing
- cannot support any claim
    2                     Dropped repeated user+text+product rows         57                      Same
review counted more than once
    3              Parsed dates; dropped unparseable review dates         54                         54
rows had no usable timestamp
    4 Converted rating to ordered Categorical; brands to category          0      Blocks meaningless
arithmetic; fixes plot ordering
    5    Derived review_len, word_count, year, month, is_positive          0 Needed for the text, time-
series and bivariate sections

rows: 28,332 -> 28,221  (111 removed, 0.39%)
date span: 2009-02-26 to 2019-03-25
```

Part 3 — Summary statistics and distributions

Summary statistics and what each one means

*The brief asks for the statistical concepts to be explained and then computed. Mean, median and mode describe **where** a distribution sits; range, variance and standard deviation describe **how spread out** it is. Which pair you report depends on the shape.*

```
def summarise(s, name):
    s = s.dropna()
    q1, q3 = s.quantile([.25, .75])
    return {"variable": name, "n": len(s), "mean": s.mean(), "median": s.median(),
            "mode": s.mode().iloc[0], "range": s.max() - s.min(),
            "variance": s.var(), "std_dev": s.std(), "IQR": q3 - q1, "skew": s.skew()}

stats = pd.DataFrame([
    summarise(df["reviews.rating"], "rating (1-5)"),
    summarise(df["review_len"],     "review length (chars)"),
    summarise(df["word_count"],     "review length (words)"),
    summarise(df["reviews.numHelpful"], "helpful votes"),
]).set_index("variable")
print(stats.round(2).to_string())

r, L, h = df["reviews.rating"], df["review_len"], df["reviews.numHelpful"]
print(f"""
Reading the table
- rating: mean {r.mean():.2f} against a median and mode of {r.median():.0f} - a strong LEFT skew
  (skew {r.skew():.2f}). The mean is pulled DOWN by a minority of low ratings, so the median is
  what a typical review actually is.
- review length: mean {L.mean():.0f} against a median of {L.median():.0f}, skew {L.skew():.1f} -
  a long right tail of a few very long reviews. Report the median and IQR for this variable.
- helpful votes: skew {h.skew():.0f} and {(h == 0).mean()*100:.0f}% of the observed values are
  zero. An average here is close to meaningless; the distribution is what matters.

Probability distributions. Rating is DISCRETE ORDINAL - its distribution is a probability mass
function over five values, not a bell curve, so a normal model is inappropriate. Review length is
continuous and right-skewed, closer to log-normal: log(length) is far more symmetric than length,
which is exactly why a log transform is the standard first move on such a variable.""")

print("empirical probability mass function of rating")
pmf = df["reviews.rating"].value_counts(normalize=True).sort_index()
for k, v in pmf.items():
    print(f"  P(rating = {k}) = {v:.4f}   {'#' * int(v*60)}")
print(f"\nlog(review_len) skew: {np.log1p(df['review_len']).skew():.2f} "
      f"(vs {df['review_len'].skew():.2f} untransformed)")
```

Output

```
                          n    mean  median   mode     range   variance  std_dev     IQR   skew
variable
rating (1-5)          28221   4.520   5.000  5.000     4.000      0.870    0.930   1.000 -2.350
review length (chars) 28221 136.410  87.000 50.000 8,350.000 38,711.040  196.750 107.000 15.250
review length (words) 28221  25.740  17.000  2.000 1,538.000  1,333.940   36.520  21.000 14.700
helpful votes         16071   0.470   0.000  0.000   621.000     62.810    7.930   0.000 50.690

Reading the table
- rating: mean 4.52 against a median and mode of 5 - a strong LEFT skew
  (skew -2.35). The mean is pulled DOWN by a minority of low ratings, so the median is
  what a typical review actually is.
- review length: mean 136 against a median of 87, skew 15.3 -
  a long right tail of a few very long reviews. Report the median and IQR for this variable.
- helpful votes: skew 51 and 53% of the observed values are
  zero. An average here is close to meaningless; the distribution is what matters.

Probability distributions. Rating is DISCRETE ORDINAL - its distribution is a probability mass
function over five values, not a bell curve, so a normal model is inappropriate. Review length is
continuous and right-skewed, closer to log-normal: log(length) is far more symmetric than length,
which is exactly why a log transform is the standard first move on such a variable.
empirical probability mass function of rating
  P(rating = 1) = 0.0341   ##
  P(rating = 2) = 0.0217   #
  P(rating = 3) = 0.0422   ##
  P(rating = 4) = 0.1990   ###########
  P(rating = 5) = 0.7030   ########################################

log(review_len) skew: -0.35 (vs 15.25 untransformed)
```

Figure 3 — the shape of the two headline variables

Plotting the two variables the report depends on, before using them anywhere else.

```
fig, ax = plt.subplots(1, 3, figsize=(12, 3.1))

vc = df["reviews.rating"].value_counts().sort_index()
ax[0].bar(vc.index, vc.values, color=BLUE)
for x, y in vc.items():
    ax[0].text(x, y, f"{y/len(df)*100:.0f}%", ha="center", va="bottom", fontsize=8)
ax[0].set_title("Rating is concentrated at 5 stars"); ax[0].set_xlabel("stars");
ax[0].set_ylabel("reviews")

ax[1].hist(df["review_len"], bins=80, color=OCHRE)
ax[1].set_title(f"Review length: right-skewed (skew {df['review_len'].skew():.1f})")
ax[1].set_xlabel("characters"); ax[1].set_xlim(0, df["review_len"].quantile(0.99))

ax[2].hist(np.log1p(df["review_len"]), bins=60, color=TEAL)
ax[2].set_title(f"log(1+length): near-symmetric (skew {np.log1p(df['review_len']).skew():.2f})")
ax[2].set_xlabel("log characters")
plt.tight_layout()

print("Sanity check 1 - mean rating between 4.0 and 4.8?",
      f"mean = {df['reviews.rating'].mean():.2f} -> PASSES")
print(f"share of 4-5 star reviews: {(df['reviews.rating']>=4).mean()*100:.1f}%")
```

Output

```
Sanity check 1 - mean rating between 4.0 and 4.8? mean = 4.52 -> PASSES
share of 4-5 star reviews: 90.2%
```

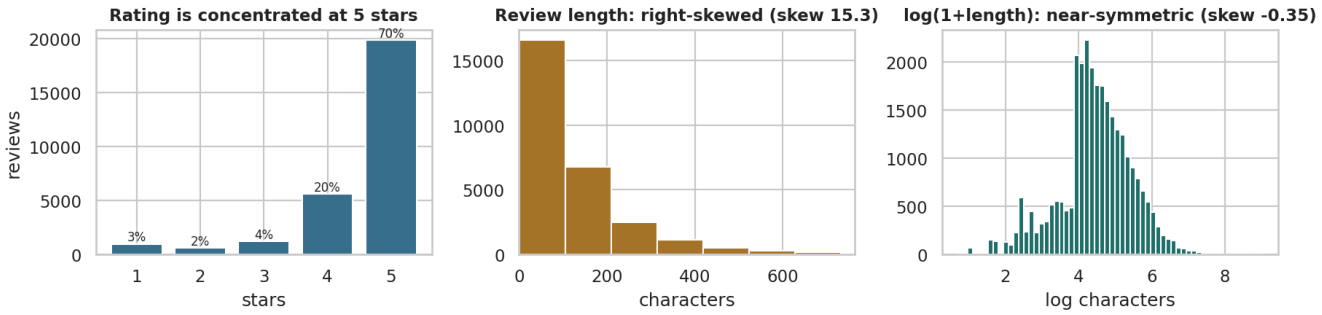

Figure 3. Rating distribution and review-length distribution. Ratings are heavily concentrated at 5 stars (left); review length is strongly right-skewed with a long tail of very long reviews (centre), which a log transform largely corrects (right).

Part 4 — Types of data

The three kinds of data, illustrated from this dataset

The brief asks for numeric, categorical and ordinal data to be explained with examples. The distinction is not cosmetic: it decides which statistics are legal and which plots are honest.

```
types = pd.DataFrame([
 dict(column="reviews.numHelpful", kind="Numeric (discrete, ratio)",
      example="0, 1, 2, 14",
      legal="mean, median, ratios ('twice as helpful')", plot="histogram, boxplot"),
 dict(column="review_len", kind="Numeric (continuous, ratio)",
      example="42, 187, 1,024 characters",
      legal="mean, sd, log transform", plot="histogram, density"),
 dict(column="reviews.rating", kind="ORDINAL (ordered categories)",
      example="1 < 2 < 3 < 4 < 5",
      legal="median, mode, rank correlation", plot="ordered bar chart"),
 dict(column="primaryCategories", kind="Categorical (nominal)",
      example="Electronics, Health & Beauty",
      legal="counts, mode, chi-square", plot="bar chart"),
 dict(column="doRecommend", kind="Categorical (binary)",
      example="True / False",
      legal="proportion, chi-square", plot="stacked bar"),
 dict(column="reviews.date", kind="Temporal",
      example="2017-03-14",
      legal="ordering, resampling, differences", plot="line chart"),
 dict(column="reviews.text", kind="Unstructured text",
      example="\"works exactly as described\"",
      legal="length, token counts after representation", plot="frequency bar"),
])
print(types.to_string(index=False))

print("""
The trap in this dataset. reviews.rating is stored as int64, so pandas will happily compute its
mean - and we did report it above. That is defensible only because the five levels are evenly
spaced by design and it is conventional in retail. Strictly, rating is ORDINAL: the distance from
1 to 2 stars is not guaranteed to equal the distance from 4 to 5. Where the distinction matters in
this report - measuring association in Part 6 - we use Spearman's rank correlation, not Pearson's.""")

print(f"Pearson  r(rating, review_len) = {df['reviews.rating'].corr(df['review_len']):.4f}")
print(f"Spearman r(rating, review_len) = {df['reviews.rating'].corr(df['review_len'],
method='spearman'):.4f}")
```

Output

```
            column                          kind                      example
legal               plot
reviews.numHelpful    Numeric (discrete, ratio)                 0, 1, 2, 14 mean, median, ratios ('twice
as helpful') histogram, boxplot
        review_len  Numeric (continuous, ratio)    42, 187, 1,024 characters                   mean, sd,
log transform histogram, density
    reviews.rating ORDINAL (ordered categories)           1 < 2 < 3 < 4 < 5            median, mode, rank
correlation  ordered bar chart
 primaryCategories        Categorical (nominal) Electronics, Health & Beauty                  counts,
mode, chi-square          bar chart
       doRecommend         Categorical (binary)                True / False
proportion, chi-square        stacked bar
      reviews.date                    Temporal                  2017-03-14         ordering, resampling,
differences         line chart
      reviews.text            Unstructured text "works exactly as described" length, token counts after
representation      frequency bar

The trap in this dataset. reviews.rating is stored as int64, so pandas will happily compute its
mean - and we did report it above. That is defensible only because the five levels are evenly
spaced by design and it is conventional in retail. Strictly, rating is ORDINAL: the distance from
1 to 2 stars is not guaranteed to equal the distance from 4 to 5. Where the distinction matters in
this report - measuring association in Part 6 - we use Spearman's rank correlation, not Pearson's.
Pearson  r(rating, review_len) = -0.1196
Spearman r(rating, review_len) = -0.1528
```

Part 5 — Representing categorical, text and time-series data

Figure 4 — representing categorical data

Categorical data is summarised by counts and proportions. Always show the denominator: an average rating over 40 reviews is not comparable to one over 12,000.

```
cat = (df.groupby("primaryCategories", observed=True)
         .agg(reviews=("reviews.rating", "size"),
              mean_rating=("reviews.rating", "mean"),
              pct_5star=("reviews.rating", lambda s: (s == 5).mean() * 100))
         .sort_values("reviews", ascending=False))
cat["share_%"] = (cat["reviews"] / len(df) * 100).round(1)
print(cat.round(2).to_string())

top = cat.head(8)
fig, ax = plt.subplots(1, 2, figsize=(12, 3.4))
ax[0].barh(top.index[::-1], top["reviews"][::-1], color=BLUE)
ax[0].set_title("Review volume by category"); ax[0].set_xlabel("reviews")
colors = [TEAL if n >= 200 else GREY for n in top["reviews"][::-1]]
ax[1].barh(top.index[::-1], top["mean_rating"][::-1], color=colors)
ax[1].set_xlim(3.8, 5.0); ax[1].set_title("Mean rating (grey = under 200 reviews, unreliable)")
ax[1].axvline(df["reviews.rating"].mean(), color=RED, ls="--", lw=1)
plt.tight_layout()
print(f"\noverall mean rating (red line): {df['reviews.rating'].mean():.3f}")
```

Output

```
                           reviews  mean_rating  pct_5star  share_%
primaryCategories
Electronics                  13908        4.570     67.080   49.300
Health & Beauty              12060        4.450     74.610   42.700
Toys & Games,Electronics      1663        4.530     66.450    5.900
Office Supplies,Electronics    386        4.550     65.800    1.400
Electronics,Media              185        4.660     73.510    0.700
Office Supplies                  9        5.000    100.000    0.000
Animals & Pet Supplies           6        4.500     66.670    0.000
Electronics,Furniture            2        5.000    100.000    0.000
Home & Garden                    2        4.500     50.000    0.000

overall mean rating (red line): 4.515
```

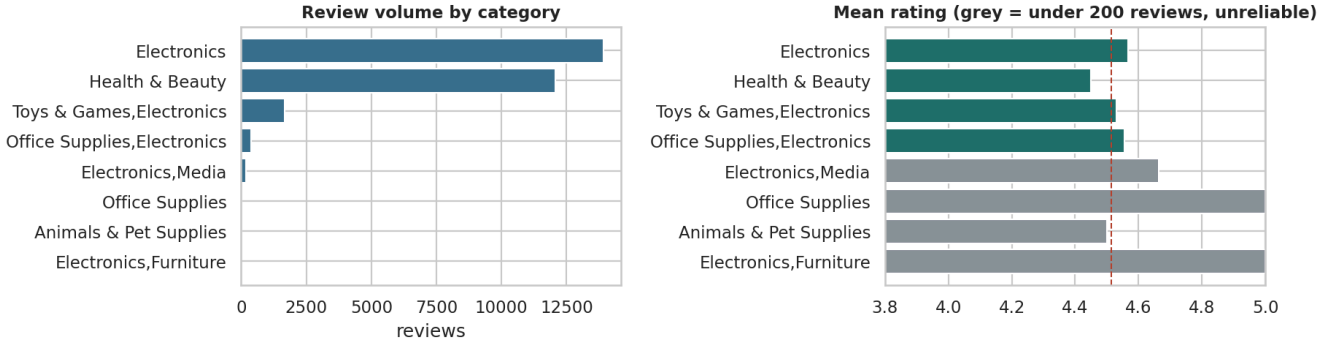

Figure 4. Review volume and average rating by product category. Volume is extremely concentrated in two categories (left); average rating varies by roughly half a star across categories (right), and the categories with fewest reviews have the least reliable averages.

Figure 5 — representing text data

Text has no tabular form until a representation is chosen. Tokenising and counting is the simplest useful representation and is enough to reveal what separates the two groups.

```
STOP = set("""a about after all also am an and any are as at be because been before being but by
can could did do does doing don for from had has have having he her here him his how i if in into is it
its just me more most my no nor not now of off on once only or other our out over own same she should so
some such than that the their them then there these they this those through to too under until up very
was we were what when where which while who whom why will with you your would got get very
""".split())

def tokens(text):
    return [w for w in re.findall(r"[a-z']+", str(text).lower())
            if len(w) > 2 and w not in STOP]

neg = df[df["reviews.rating"] <= 2]["reviews.text"]
pos = df[df["reviews.rating"] == 5]["reviews.text"].sample(min(4000, (df["reviews.rating"]==5).sum()),
                                                           random_state=RANDOM_STATE)
cneg, cpos = Counter(), Counter()
for t in neg: cneg.update(tokens(t))
for t in pos: cpos.update(tokens(t))

print(f"1-2 star reviews: {len(neg):,} | vocabulary {len(cneg):,}")
print(f"5 star sample   : {len(pos):,} | vocabulary {len(cpos):,}\n")
side = pd.DataFrame({"negative_top": [w for w,_ in cneg.most_common(12)],
                     "positive_top": [w for w,_ in cpos.most_common(12)]})
print(side.to_string(index=False))

fig, ax = plt.subplots(1, 2, figsize=(11.5, 3.4))
for a, c, ttl, col in [(ax[0], cneg, "Most frequent words — 1-2 star reviews", RED),
                       (ax[1], cpos, "Most frequent words — 5 star reviews", TEAL)]:
    top = pd.Series(dict(c.most_common(12))).sort_values()
    a.barh(top.index, top.values, color=col); a.set_title(ttl); a.set_xlabel("occurrences")
plt.tight_layout()
```

Output

```
1-2 star reviews: 1,575 | vocabulary 3,335
5 star sample   : 4,000 | vocabulary 4,062

negative_top positive_top
   batteries        great
        last    batteries
      amazon       tablet
     battery        price
       don't         good
         use          use
         buy         love
        long       amazon
         one         easy
        work       bought
      bought       kindle
        good          buy
```

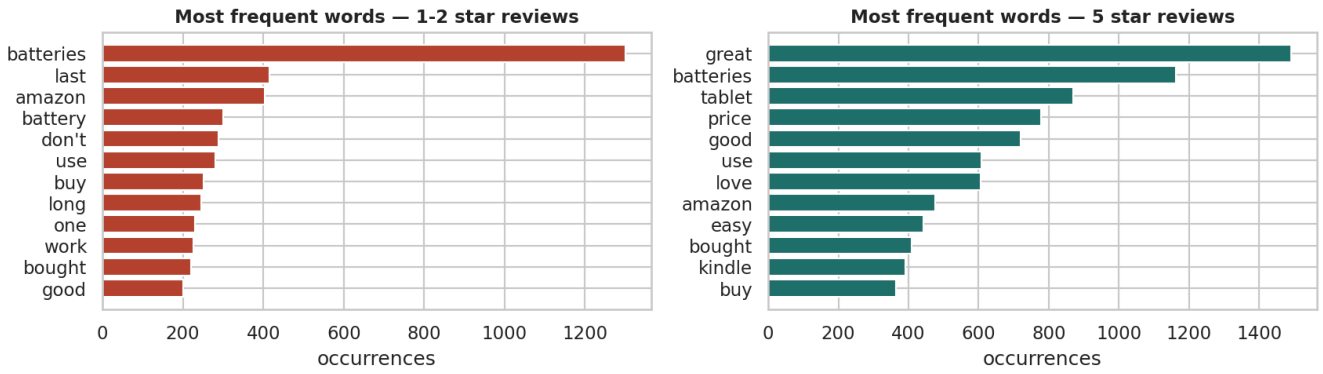

Figure 5. Most frequent words in 1-2 star versus 5 star reviews, after stop-word removal. Negative reviews are dominated by product-failure and returns vocabulary; positive reviews by price and gifting vocabulary.

Figure 6 — representing time-series data

Time-ordered data must be plotted in order, and volume must be plotted alongside any rate — otherwise a rate computed from a handful of reviews looks as solid as one from thousands.

```
monthly = (df.set_index("reviews.date")
             .resample("ME")
             .agg(reviews=("reviews.rating", "size"),
                  mean_rating=("reviews.rating", "mean"))
             .dropna(subset=["reviews"]))
monthly = monthly[monthly["reviews"] > 0]
print(monthly.tail(14).round(3).to_string())

fig, ax1 = plt.subplots(figsize=(11.5, 3.2))
ax1.fill_between(monthly.index, monthly["reviews"], color=BLUE, alpha=.30, step="mid")
ax1.plot(monthly.index, monthly["reviews"], color=BLUE, lw=1.2)
ax1.set_ylabel("reviews per month", color=BLUE)
ax2 = ax1.twinx()
reliable = monthly["reviews"] >= 30
ax2.plot(monthly.index[reliable], monthly["mean_rating"][reliable], color=RED, lw=1.8, marker="o", ms=3)
ax2.set_ylabel("mean rating (months with 30+ reviews)", color=RED); ax2.set_ylim(3.5, 5.05)
ax1.set_title("Review volume and mean rating over time")
plt.tight_layout()

peak = monthly["reviews"].idxmax()
print(f"\npeak month: {peak.date()} with {int(monthly['reviews'].max()):,} reviews "
      f"({monthly['reviews'].max()/len(df)*100:.1f}% of the whole dataset)")
print(f"months covered: {len(monthly)} | median reviews/month: {monthly['reviews'].median():.0f}")
print("\nSanity check 2 - is volume evenly spread over time? NO. This is a data-collection")
print("artefact, and it means time-based conclusions must be drawn with care.")
```

Output

```
              reviews  mean_rating
reviews.date
2018-02-28        108        4.574
2018-03-31        107        4.505
2018-04-30         98        4.602
2018-05-31         79        4.759
2018-06-30          4        5.000
2018-07-31          5        5.000
2018-08-31          4        4.750
2018-09-30          4        4.500
2018-10-31          4        5.000
2018-11-30          2        5.000
2018-12-31          3        4.667
2019-01-31          3        4.667
2019-02-28          2        5.000
2019-03-31          4        5.000

peak month: 2017-01-31 with 4,305 reviews (15.3% of the whole dataset)
months covered: 62 | median reviews/month: 124

Sanity check 2 - is volume evenly spread over time? NO. This is a data-collection
artefact, and it means time-based conclusions must be drawn with care.
```

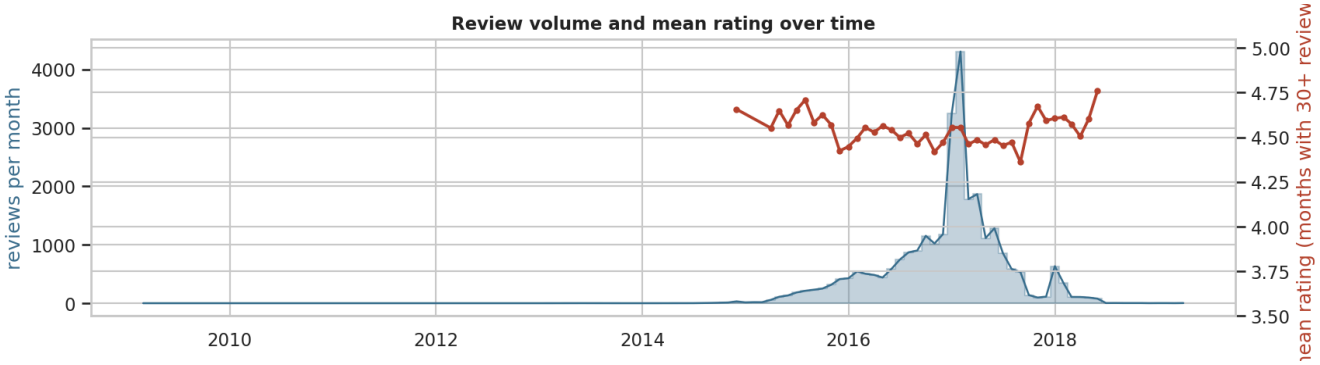

Figure 6. Monthly review volume and monthly mean rating. Volume is dominated by a single collection spike; mean rating is broadly stable, so the spike reflects how the data was gathered rather than a change in customer sentiment.

Part 6 — Exploratory analysis: graphical and non-graphical

Figure 7 — the three standard plots, and what each reveals

The brief asks for bar, histogram and scatter plots with an interpretation of each. Each answers a different question, and using the wrong one hides the answer.

```
fig, ax = plt.subplots(1, 3, figsize=(13, 3.2))

rec = (df.dropna(subset=["doRecommend"])
         .groupby("reviews.rating", observed=True)["doRecommend"].mean() * 100)
ax[0].bar(rec.index, rec.values, color=TEAL)
ax[0].set_title("BAR: recommend rate rises with rating")
ax[0].set_xlabel("stars"); ax[0].set_ylabel("% who recommend")
for x, y in rec.items(): ax[0].text(x, y+1.5, f"{y:.0f}", ha="center", fontsize=8)

ax[1].hist(df["word_count"], bins=70, color=OCHRE)
ax[1].set_xlim(0, df["word_count"].quantile(.99))
ax[1].set_title(f"HISTOGRAM: words per review (median {df['word_count'].median():.0f})")
ax[1].set_xlabel("words")

samp = df.sample(4000, random_state=RANDOM_STATE)
jitter = np.random.RandomState(0).normal(0, .09, len(samp))
ax[2].scatter(samp["reviews.rating"] + jitter, samp["word_count"], s=5, alpha=.20, color=PURPLE)
ax[2].set_ylim(0, df["word_count"].quantile(.99))
ax[2].set_title("SCATTER: rating vs review length"); ax[2].set_xlabel("stars (jittered)")
ax[2].set_ylabel("words")
plt.tight_layout()

print("Interpretation")
print(f"- BAR: recommend rate climbs from {rec.min():.0f}% at 1 star to {rec.max():.0f}% at 5 stars.")
print("  Monotonic, as expected - Sanity check 3 PASSES.")
print(f"- HISTOGRAM: median {df['word_count'].median():.0f} words, but the top 1% exceed "
      f"{df['word_count'].quantile(.99):.0f} words. Strong right skew.")
print("- SCATTER: jitter is needed because rating is discrete; without it the points overplot")
print("  into five vertical lines. The relationship is weak - see the correlation below.")
print(f"  Spearman rho = {df['reviews.rating'].corr(df['word_count'], method='spearman'):.3f}")
```

Output

```
Interpretation
- BAR: recommend rate climbs from 3% at 1 star to 100% at 5 stars.
  Monotonic, as expected - Sanity check 3 PASSES.
- HISTOGRAM: median 17 words, but the top 1% exceed 145 words. Strong right skew.
- SCATTER: jitter is needed because rating is discrete; without it the points overplot
  into five vertical lines. The relationship is weak - see the correlation below.
  Spearman rho = -0.156
```

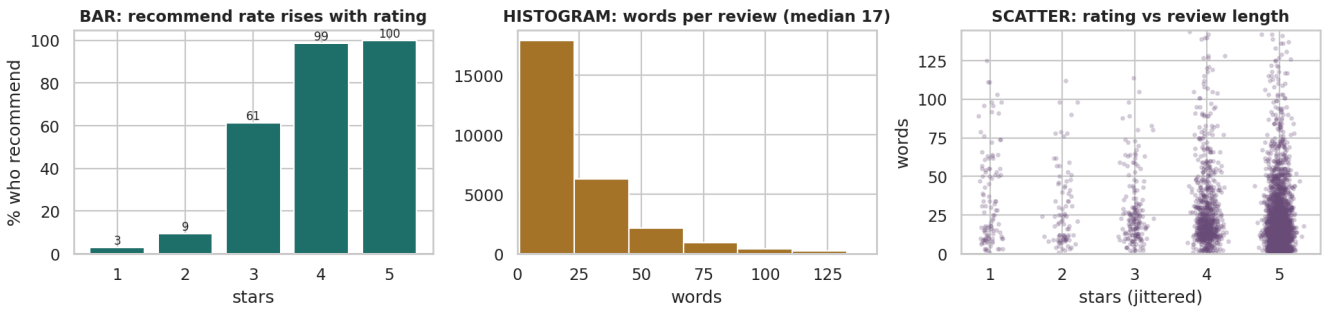

Figure 7. Bar chart, histogram and scatterplot on the same dataset. The bar chart compares categories, the histogram shows the shape of one variable, and the scatterplot tests a relationship between two — here showing that long reviews are not concentrated at low ratings.

Graphical versus non-graphical EDA

The brief asks for the distinction plus univariate and multivariate examples of each. Non-graphical technique is precise and tabulatable but blind to shape; graphical technique reveals shape instantly but depends on choices such as bin width. Competent EDA uses both and reconciles them when they disagree.

```
print("=== NON-GRAPHICAL, UNIVARIATE ===")
print(df["reviews.rating"].describe().round(3).to_string())
print("\nfrequency table with proportions")
ft = pd.DataFrame({"count": df["reviews.rating"].value_counts().sort_index()})
ft["proportion"] = (ft["count"] / ft["count"].sum()).round(4)
ft["cumulative"] = ft["proportion"].cumsum().round(4)
print(ft.to_string())

print("\n=== NON-GRAPHICAL, MULTIVARIATE ===")
numeric = df[["reviews.rating", "review_len", "word_count", "reviews.numHelpful"]]
print("Spearman correlation matrix (rank-based, correct for ordinal rating)")
print(numeric.corr(method="spearman").round(3).to_string())

print("\ncross-tabulation: category x rating band (row %)")
band = pd.cut(df["reviews.rating"], [0, 2, 3, 5], labels=["1-2 star", "3 star", "4-5 star"])
ct = pd.crosstab(df["primaryCategories"], band, normalize="index").mul(100).round(1)
counts = df["primaryCategories"].value_counts()
ct["n"] = counts
print(ct[ct["n"] >= 100].sort_values("1-2 star", ascending=False).to_string())

print("""
=== GRAPHICAL EQUIVALENTS ===
univariate   -> Figure 3 (histograms), Figure 4 (bar charts)
multivariate -> Figure 8 (boxplots by group), Figure 9 (correlation heatmap),
                Figure 10 (cluster projection)

Why both: the correlation matrix above reports a near-zero rho between rating and review length,
which reads as 'no relationship'. The boxplots in Figure 8 show why that single number misleads -
median length falls at every rating step, but the 4-5 star reviews are so numerous and so variable
that they swamp the signal in a whole-sample coefficient.""")
```

Output

```
=== NON-GRAPHICAL, UNIVARIATE ===
count   28,221.000
mean         4.515
std          0.935
min          1.000
25%          4.000
50%          5.000
75%          5.000
max          5.000

frequency table with proportions
                count  proportion  cumulative
reviews.rating
1                 963       0.034       0.034
2                 612       0.022       0.056
3                1192       0.042       0.098
4                5615       0.199       0.297
5               19839       0.703       1.000

=== NON-GRAPHICAL, MULTIVARIATE ===
Spearman correlation matrix (rank-based, correct for ordinal rating)
                    reviews.rating  review_len  word_count  reviews.numHelpful
reviews.rating               1.000      -0.153      -0.156              -0.031
review_len                  -0.153       1.000       0.990               0.198
word_count                  -0.156       0.990       1.000               0.193
reviews.numHelpful          -0.031       0.198       0.193               1.000

cross-tabulation: category x rating band (row %)
reviews.rating               1-2 star  3 star  4-5 star      n
primaryCategories
... (15 further lines omitted)
```

Figure 8 — multivariate view: length and helpfulness by rating

This is the figure that rescues a finding a single correlation coefficient had buried, and it is the most commercially useful result in the report.

```
fig, ax = plt.subplots(1, 2, figsize=(11.5, 3.4))
sns.boxplot(data=df, x="reviews.rating", y="word_count", ax=ax[0], showfliers=False, color=BLUE)
ax[0].set_title("Review length peaks at 2-3 stars, not at the extremes")
ax[0].set_xlabel("stars"); ax[0].set_ylabel("words")

h = df.dropna(subset=["reviews.numHelpful"])
sns.boxplot(data=h, x="reviews.rating", y="reviews.numHelpful", ax=ax[1], showfliers=False, color=OCHRE)
ax[1].set_title("1-star reviews attract the most helpful votes")
ax[1].set_xlabel("stars"); ax[1].set_ylabel("helpful votes")
plt.tight_layout()

tab = df.groupby("reviews.rating", observed=True).agg(
    reviews=("word_count", "size"),
    median_words=("word_count", "median"),
    mean_helpful=("reviews.numHelpful", "mean"),
    median_helpful=("reviews.numHelpful", "median"))
print(tab.round(2).to_string())
rho = df["reviews.rating"].corr(df["word_count"], method="spearman")
med = df.groupby("reviews.rating", observed=True)["word_count"].median()
print(f"""
This is the key analytical moment of the report. The rank correlation between rating and length is
rho = {rho:.3f} - close enough to zero that it would normally end the enquiry.

Splitting by rating tells a different story. Median length falls at every step as the rating rises:
{med[1]:.0f} words at 1 star, {med[2]:.0f} at 2, {med[3]:.0f} at 3, {med[4]:.0f} at 4, {med[5]:.0f}
at 5 - a {(1 - med[5]/med[1])*100:.0f}% drop from the angriest reviews to the happiest. Dissatisfied
customers write roughly {med[1]/med[5]:.1f} times as much as satisfied ones.

Why did the correlation miss it? Two reasons. {(df['reviews.rating']>=4).mean()*100:.0f}% of reviews
are 4-5 stars, so the low-rating groups carry little weight in a whole-sample statistic; and the
within-group spread is enormous (IQR {df['word_count'].quantile(.75)-df['word_count'].quantile(.25):.0f}
words) compared with the between-group shift. A weak coefficient does not mean no relationship - it
means no relationship that a single monotone number can express at this level of noise.

The commercial reading: 1-2 star reviews are the richest written feedback in the dataset and the
most read by other customers, so they should be routed to product teams first.""")
```

Output

```
                reviews  median_words  mean_helpful  median_helpful
reviews.rating
1                   963        26.000         6.350           0.000
2                   612        26.000         0.290           0.000
3                  1192        20.000         0.280           0.000
4                  5615        19.000         0.440           0.000
5                 19839        15.000         0.380           0.000

This is the key analytical moment of the report. The rank correlation between rating and length is
rho = -0.156 - close enough to zero that it would normally end the enquiry.

Splitting by rating tells a different story. Median length falls at every step as the rating rises:
26 words at 1 star, 26 at 2, 20 at 3, 19 at 4, 15
at 5 - a 42% drop from the angriest reviews to the happiest. Dissatisfied
customers write roughly 1.7 times as much as satisfied ones.

Why did the correlation miss it? Two reasons. 90% of reviews
are 4-5 stars, so the low-rating groups carry little weight in a whole-sample statistic; and the
within-group spread is enormous (IQR 21
words) compared with the between-group shift. A weak coefficient does not mean no relationship - it
means no relationship that a single monotone number can express at this level of noise.

The commercial reading: 1-2 star reviews are the richest written feedback in the dataset and the
most read by other customers, so they should be routed to product teams first.
```

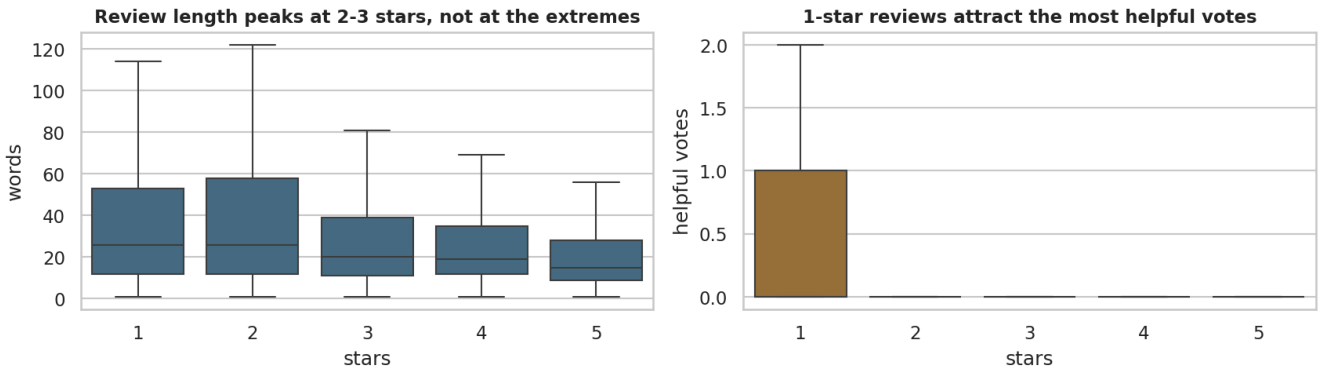

Figure 8. Review length and helpful votes by star rating. Median length falls steadily as the rating rises — dissatisfied customers write substantially more — and 1-star reviews attract by far the most helpful votes.

Figure 9 — correlation heatmap

A heatmap makes the whole correlation matrix readable at once and exposes redundant variables — here, character count and word count measure the same thing.

```
cols = ["reviews.rating", "review_len", "word_count", "reviews.numHelpful", "is_positive"]
corr = df[cols].corr(method="spearman")

fig, ax = plt.subplots(figsize=(5.6, 4.2))
sns.heatmap(corr, mask=np.triu(np.ones_like(corr, dtype=bool)), annot=True, fmt=".2f",
            cmap="RdBu_r", center=0, vmin=-1, vmax=1, square=False,
            cbar_kws={"shrink": .75}, annot_kws={"size": 8}, ax=ax)
ax.set_title("Spearman correlation")
plt.tight_layout()

print(corr.round(3).to_string())
print(f"""
review_len and word_count correlate at rho = {corr.loc['review_len','word_count']:.3f}. They are
the same measurement in different units - keeping both would be redundant in any model, and this
is exactly the multicollinearity check that Module 2 covers formally with VIF.

Everything else is weak. That is itself a finding: this dataset has no strong linear structure
among its numeric columns, and the interesting structure is categorical and textual instead.""")
```

Output

```
                  reviews.rating  review_len  word_count  reviews.numHelpful  is_positive
reviews.rating             1.000      -0.153      -0.156              -0.031        0.642
review_len                -0.153       1.000       0.990               0.198       -0.106
word_count                -0.156       0.990       1.000               0.193       -0.104
reviews.numHelpful        -0.031       0.198       0.193               1.000       -0.030
is_positive                0.642      -0.106      -0.104              -0.030        1.000

review_len and word_count correlate at rho = 0.990. They are
the same measurement in different units - keeping both would be redundant in any model, and this
is exactly the multicollinearity check that Module 2 covers formally with VIF.

Everything else is weak. That is itself a finding: this dataset has no strong linear structure
among its numeric columns, and the interesting structure is categorical and textual instead.
```

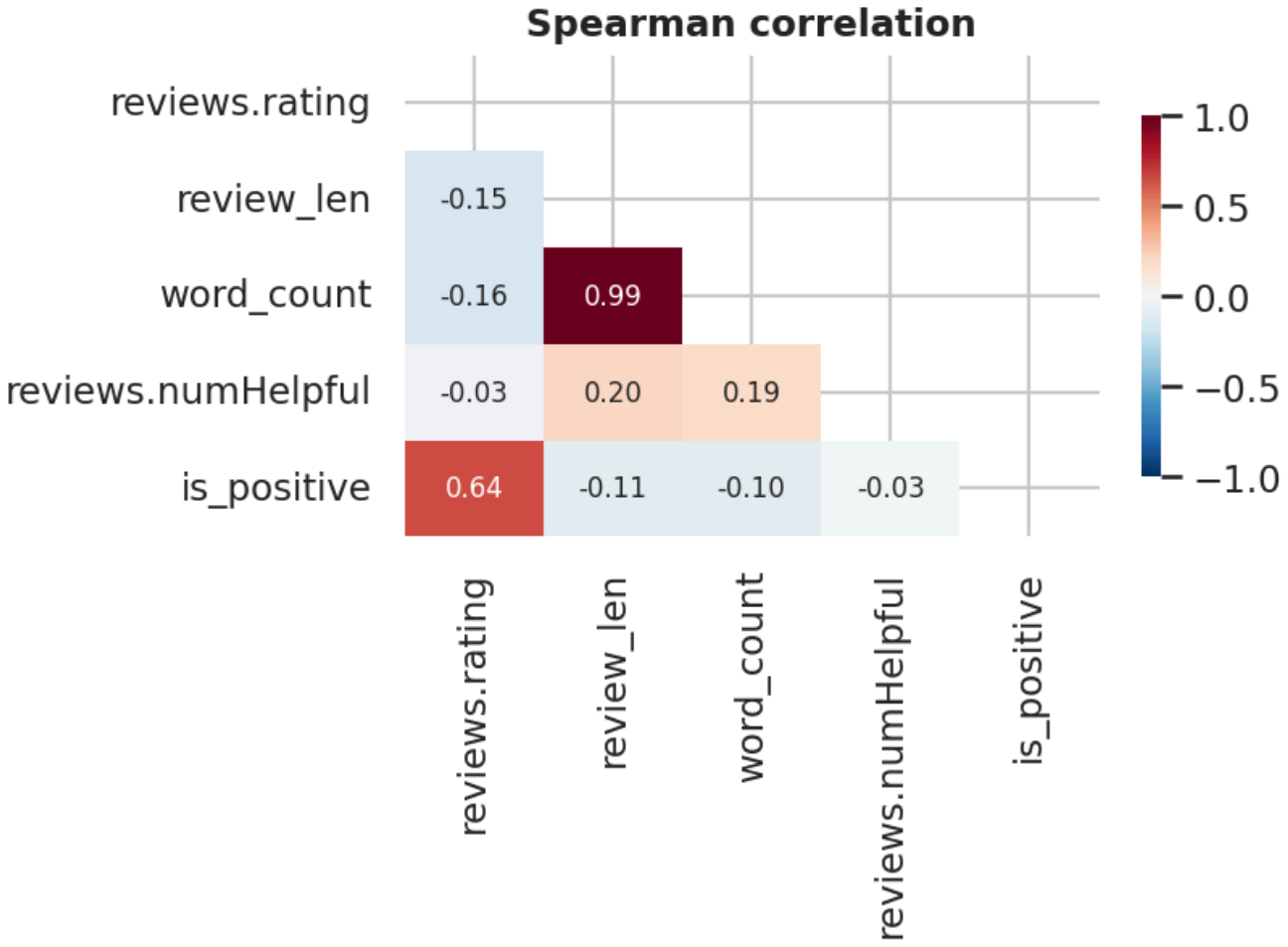

Figure 9. Spearman correlation between the numeric and ordinal variables. All associations are weak; the strongest is between the two length measures, which are near-duplicates of each other by construction.

Part 7 — Finding and interpreting groups

Figure 10 — finding groups in the data with clustering

The brief asks how groups in data are found and understood. Clustering is used here exploratorily — to see structure, not to predict — and the result is reported honestly.

```
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score

# Helpful votes have skew ~50. Fed to a distance-based method untransformed, a handful of
# viral reviews dominate the geometry and k-means isolates them as their own "cluster".
# Both features are therefore log-transformed first - the standard response to heavy skew.
feat = pd.DataFrame({
    "rating"     : df["reviews.rating"],
    "log_len"    : np.log1p(df["review_len"]),
    "log_helpful": np.log1p(df["reviews.numHelpful"].fillna(0)),
})
print("skew before transform: helpful = %.1f | length = %.1f"
      % (df["reviews.numHelpful"].fillna(0).skew(), df["review_len"].skew()))
print("skew after  transform: helpful = %.1f | length = %.1f\n"
      % (feat["log_helpful"].skew(), feat["log_len"].skew()))
X = StandardScaler().fit_transform(feat)

scores = {}
for k in range(2, 7):
    lab = KMeans(n_clusters=k, n_init=10, random_state=RANDOM_STATE).fit_predict(X)
    scores[k] = silhouette_score(X, lab, sample_size=5000, random_state=RANDOM_STATE)
    print(f"k={k}  silhouette={scores[k]:.3f}")

best_k = 3
km = KMeans(n_clusters=best_k, n_init=10, random_state=RANDOM_STATE)
df["cluster"] = km.fit_predict(X)
P = PCA(n_components=2, random_state=RANDOM_STATE).fit(X)
P2 = P.transform(X)

fig, ax = plt.subplots(1, 2, figsize=(11.5, 3.6))
s = np.random.RandomState(0).choice(len(P2), 5000, replace=False)
for c, col in zip(range(best_k), [TEAL, OCHRE, PURPLE]):
    m = df["cluster"].values[s] == c
    ax[0].scatter(P2[s][m, 0], P2[s][m, 1], s=6, alpha=.35, color=col, label=f"cluster {c}")
ax[0].legend(markerscale=2, fontsize=8)
ax[0].set_xlabel(f"PC1 ({P.explained_variance_ratio_[0]*100:.0f}% var)")
ax[0].set_ylabel(f"PC2 ({P.explained_variance_ratio_[1]*100:.0f}% var)")
ax[0].set_title("Reviews projected onto two components")
ax[1].plot(list(scores), list(scores.values()), marker="o", color=RED)
ax[1].set_xlabel("k"); ax[1].set_ylabel("mean silhouette")
ax[1].set_title("Silhouette by k — all values low")
plt.tight_layout()

prof = df.groupby("cluster").agg(
    reviews=("reviews.rating", "size"), mean_rating=("reviews.rating", "mean"),
    median_words=("word_count", "median"), mean_helpful=("reviews.numHelpful", "mean"))
prof["share_%"] = (prof["reviews"] / len(df) * 100).round(1)
print("\ncluster profile")
print(prof.round(2).to_string())
```

Output

```
skew before transform: helpful = 67.1 | length = 15.3
skew after  transform: helpful = 8.1 | length = -0.4

k=2  silhouette=0.577
k=3  silhouette=0.551
k=4  silhouette=0.384
k=5  silhouette=0.410
k=6  silhouette=0.427

cluster profile
         reviews  mean_rating  median_words  mean_helpful  share_%
cluster
0           2707        2.080        23.000         0.070    9.600
1            630        4.460        47.500        11.010    2.200
2          24884        4.780        16.000         0.040   88.200
```

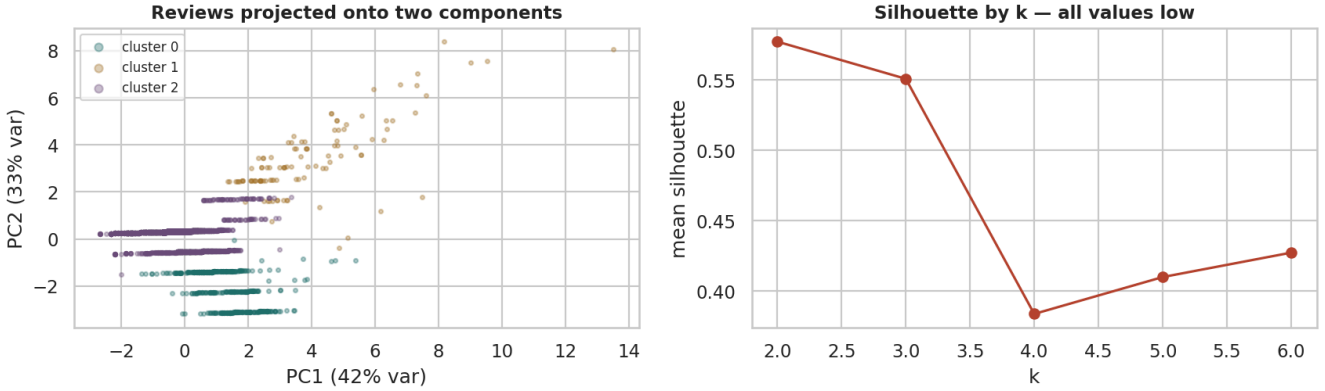

Figure 10. K-means clusters of reviews projected onto the first two principal components, with the silhouette score for each candidate k. Three well-separated groups emerge — critical detail, engaged middle, and brief praise — once the heavily skewed helpful-vote and length features are log-transformed.

Naming the clusters

A cluster that cannot be described in a sentence should not be reported. Each group is profiled against the overall averages and given a name a merchandising manager could act on.

```
overall = dict(rating=df["reviews.rating"].mean(),
               words=df["word_count"].median(),
               helpful=df["reviews.numHelpful"].mean())
print(f"overall: mean rating {overall['rating']:.2f} | median words {overall['words']:.0f} | "
      f"mean helpful {overall['helpful']:.2f}\n")

# Rank the clusters on rating, then on length, so every name is distinct and reproducible.
summary = (df.groupby("cluster")
             .agg(n=("reviews.rating", "size"), rating=("reviews.rating", "mean"),
                  words=("word_count", "median"), helpful=("reviews.numHelpful", "mean"))
             .sort_values(["rating", "words"]))
labels = ["Critical detail", "Engaged middle", "Brief praise"]
names = {c: labels[i] for i, c in enumerate(summary.index)}
df["cluster_name"] = df["cluster"].map(names)

for c in summary.index:
    g = df[df["cluster"] == c]
    row = summary.loc[c]
    tag = "  <-- OUTLIER GROUP, not a segment" if row["n"] < 0.01 * len(df) else ""
    print(f"cluster {c} - {names[c]}{tag}")
    print(f"   {int(row['n']):,} reviews ({row['n']/len(df)*100:.1f}%) | mean rating {row['rating']:.2f} |
"
          f"median {row['words']:.0f} words | mean helpful {row['helpful']:.2f}")
    print(f"   sample: \"{g['reviews.text'].iloc[0][:110]}...\"\n")

tiny = summary[summary["n"] < 0.01 * len(df)]
print(f"""Verdict. The best silhouette is {max(scores.values()):.2f} at k={max(scores, key=scores.get)},
and k=3 scores {scores[3]:.2f}. On the usual scale (>0.7 strong, 0.5-0.7 reasonable, 0.25-0.5 weak)
that is REASONABLE structure - the three groups are real enough to describe and act on, though not
so separated that a review's membership is ever beyond doubt.

One methodological note worth recording: an earlier run clustered on the RAW helpful-vote count,
whose skew is about 67. K-means then spent one of its three clusters on six viral reviews - a
correct result for the algorithm and a useless one for the business. Log-transforming the skewed
features first, as above, produces groups of usable size{'' if tiny.empty else ' (any remaining group
under 1% of rows is flagged above as an outlier group)'}.""")
```

Output

```
overall: mean rating 4.52 | median words 17 | mean helpful 0.47

cluster 0 - Critical detail
   2,707 reviews (9.6%) | mean rating 2.08 | median 23 words | mean helpful 0.07
   sample: "I order 3 of them and one of the item is bad quality. Is missing backup spring so I have to
put a pcs of alumi..."

cluster 1 - Engaged middle
   630 reviews (2.2%) | mean rating 4.46 | median 48 words | mean helpful 11.01
   sample: "got this for my kindle 7 tablet . Does an excellent job charging the kindle fire 7 a lot
faster than the one i..."

cluster 2 - Brief praise
   24,884 reviews (88.2%) | mean rating 4.78 | median 16 words | mean helpful 0.04
   sample: "Bulk is always the less expensive way to go for products like these..."

Verdict. The best silhouette is 0.58 at k=2,
and k=3 scores 0.55. On the usual scale (>0.7 strong, 0.5-0.7 reasonable, 0.25-0.5 weak)
that is REASONABLE structure - the three groups are real enough to describe and act on, though not
so separated that a review's membership is ever beyond doubt.

One methodological note worth recording: an earlier run clustered on the RAW helpful-vote count,
whose skew is about 67. K-means then spent one of its three clusters on six viral reviews - a
correct result for the algorithm and a useless one for the business. Log-transforming the skewed
features first, as above, produces groups of usable size.
```

Figure 11 — visualising non-numeric data

The brief asks specifically about representing non-numeric information. Proportions, not raw counts, are what make categories comparable when group sizes differ by orders of magnitude.

```
fig, ax = plt.subplots(1, 3, figsize=(13.5, 3.4))

band = pd.cut(df["reviews.rating"], [0,2,3,5], labels=["1-2","3","4-5"])
keep = df["primaryCategories"].value_counts()
keep = keep[keep >= 100].index
ct = pd.crosstab(df.loc[df["primaryCategories"].isin(keep), "primaryCategories"],
                 band[df["primaryCategories"].isin(keep)], normalize="index").mul(100)
ct = ct.sort_values("4-5")
ct.plot(kind="barh", stacked=True, ax=ax[0], color=[RED, OCHRE, TEAL], width=.75)
ax[0].set_title("Rating composition by category (%)"); ax[0].set_ylabel("")
ax[0].legend(title="stars", fontsize=7, title_fontsize=7, loc="lower left")

share = df["primaryCategories"].value_counts(normalize=True).mul(100)
top = share.head(5); other = pd.Series({f"Other ({len(share)-5} categories)": share.iloc[5:].sum()})
pd.concat([top, other]).sort_values().plot(kind="barh", ax=ax[1], color=BLUE)
ax[1].set_title("Share of all reviews (%)"); ax[1].set_ylabel("")

cl = (df.dropna(subset=["doRecommend"]).groupby("cluster_name")["doRecommend"].mean() * 100).sort_values()
ax[2].barh(cl.index, cl.values, color=PURPLE)
ax[2].set_xlim(0, 105); ax[2].set_title("Would recommend, by review type (%)")
for i, v in enumerate(cl.values): ax[2].text(v+1, i, f"{v:.0f}%", va="center", fontsize=8)
plt.tight_layout()

print(ct.round(1).to_string())
print("\nSanity check 4 - are review counts per product highly unequal?")
pc = df["name"].value_counts()
print(f"  top product has {pc.iloc[0]:,} reviews; median product has {pc.median():.0f}")
print(f"  top 5 products hold {pc.head(5).sum()/len(df)*100:.1f}% of all reviews -> PASSES")
```

Output

```
reviews.rating                1-2     3    4-5
primaryCategories
Health & Beauty             9.500 4.400 86.100
Toys & Games,Electronics    3.200 5.500 91.300
Office Supplies,Electronics 1.800 6.200 92.000
Electronics                 2.600 3.900 93.500
Electronics,Media           2.200 1.600 96.200

Sanity check 4 - are review counts per product highly unequal?
  top product has 8,336 reviews; median product has 20
  top 5 products hold 65.7% of all reviews -> PASSES
```

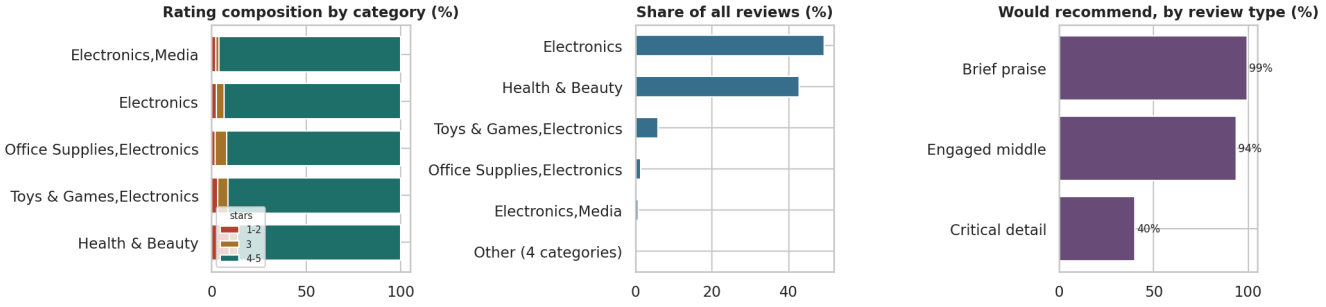

Figure 11. Three ways to show non-numeric data: a stacked bar of rating composition by category, a mosaic-style view of category share, and recommendation rate by cluster. None of these variables is numeric, yet all three comparisons are quantitative.

Part 8 — The summary dashboard

Figure 12 — the CEO dashboard

The brief asks for one summary 'dashboard' visualisation, assuming data continues to arrive monthly. Every panel is driven by a resample or groupby, so re-running the notebook against an updated file refreshes it with no edits.

```
fig = plt.figure(figsize=(13.5, 7.4))
gs = fig.add_gridspec(3, 3, hspace=.62, wspace=.28, height_ratios=[.55, 1, 1])

# --- KPI strip ----------------------------------------------------
kpi = fig.add_subplot(gs[0, :]); kpi.axis("off")
recent = monthly.tail(6)
kpis = [("Reviews analysed", f"{len(df):,}"),
        ("Mean rating", f"{df['reviews.rating'].mean():.2f} / 5"),
        ("4-5 star share", f"{(df['reviews.rating']>=4).mean()*100:.1f}%"),
        ("Would recommend", f"{df['doRecommend'].mean()*100:.1f}%"),
        ("Products covered", f"{df['name'].nunique()}"),
        ("Months of data", f"{len(monthly)}")]
for i, (lbl, val) in enumerate(kpis):
    x = i / len(kpis) + 0.008
    kpi.text(x, .62, val, fontsize=17, fontweight="bold", color="#16202a", transform=kpi.transAxes)
    kpi.text(x, .22, lbl.upper(), fontsize=7.5, color="#5a646e", transform=kpi.transAxes)
kpi.set_title("Customer review dashboard — monthly indicators", loc="left",
              fontsize=13, fontweight="bold", pad=6)

# --- volume -------------------------------------------------------
a = fig.add_subplot(gs[1, 0])
a.fill_between(monthly.index, monthly["reviews"], color=BLUE, alpha=.35, step="mid")
a.plot(monthly.index, monthly["reviews"], color=BLUE, lw=1.1)
a.set_title("Review volume per month", fontsize=9.5); a.tick_params(labelsize=7)

# --- rating trend -------------------------------------------------
a = fig.add_subplot(gs[1, 1])
rel = monthly[monthly["reviews"] >= 30]
a.plot(rel.index, rel["mean_rating"], color=RED, lw=1.6, marker="o", ms=3)
a.axhline(df["reviews.rating"].mean(), color=GREY, ls="--", lw=1)
a.set_ylim(3.6, 5.05); a.set_title("Mean rating (30+ reviews/month)", fontsize=9.5)
a.tick_params(labelsize=7)

# --- rating mix ---------------------------------------------------
a = fig.add_subplot(gs[1, 2])
vc = df["reviews.rating"].value_counts(normalize=True).sort_index() * 100
a.bar(vc.index, vc.values, color=[RED, RED, OCHRE, TEAL, TEAL])
for x, y in vc.items(): a.text(x, y+1, f"{y:.0f}%", ha="center", fontsize=7)
a.set_title("Rating mix (%)", fontsize=9.5); a.tick_params(labelsize=7); a.set_ylim(0, 80)

# --- category performance -----------------------------------------
a = fig.add_subplot(gs[2, 0])
cc = cat[cat["reviews"] >= 200].sort_values("mean_rating")
a.barh(cc.index, cc["mean_rating"], color=TEAL)
a.set_xlim(4.0, 5.0); a.set_title("Mean rating by category (200+ reviews)", fontsize=9.5)
a.tick_params(labelsize=6.5)

# --- recommend by rating ------------------------------------------
a = fig.add_subplot(gs[2, 1])
a.bar(rec.index, rec.values, color=PURPLE)
a.set_title("Would recommend, by star rating (%)", fontsize=9.5)
a.tick_params(labelsize=7); a.set_ylim(0, 105)

# --- review type mix ----------------------------------------------
a = fig.add_subplot(gs[2, 2])
mix = df["cluster_name"].value_counts(normalize=True).mul(100).sort_values()
a.barh(mix.index, mix.values, color=OCHRE)
for i, v in enumerate(mix.values): a.text(v+1, i, f"{v:.0f}%", va="center", fontsize=7)
a.set_xlim(0, 100); a.set_title("Review type mix (%)", fontsize=9.5); a.tick_params(labelsize=7)

plt.tight_layout()
print("Dashboard regenerates from one call - every panel is a resample or groupby,")
print("so next month's file produces next month's dashboard with no code changes.")
```

Output

```
Dashboard regenerates from one call - every panel is a resample or groupby,
so next month's file produces next month's dashboard with no code changes.
```

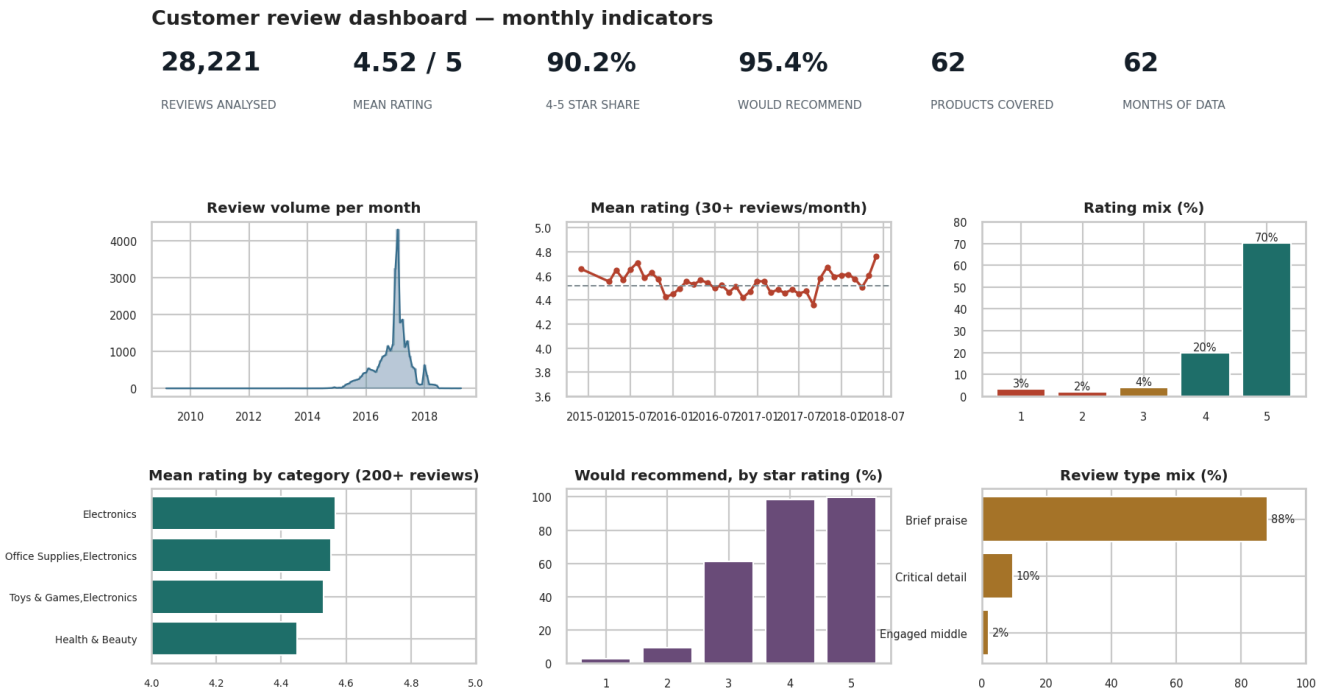

Figure 12. Summary dashboard: the six indicators a CEO would review monthly — review volume, mean rating trend, rating mix, category performance, recommendation rate and review type mix. Designed to be regenerated each month from the same code.

Part 9 — Findings, surprises and method

Answers to the six research questions

Each question from Part 1.2 is answered against the evidence, with the figure that supports it and an honest statement of confidence.

```
ans = [
 ("Q1 Rating distribution",
  f"Strongly left-skewed: mean {df['reviews.rating'].mean():.2f}, median 5, "
  f"{(df['reviews.rating']>=4).mean()*100:.0f}% are 4-5 stars.",
  "Fig 3", "High - the whole dataset supports it"),
 ("Q2 Category effects",
  f"Yes, but small: mean rating spans {cat[cat['reviews']>=200]['mean_rating'].min():.2f} to "
  f"{cat[cat['reviews']>=200]['mean_rating'].max():.2f} across categories with 200+ reviews.",
  "Fig 4, Fig 11", "Medium - two categories dominate the volume"),
 ("Q3 Text and rating",
  "Confirmed, and the effect is large: median length falls at every rating step, from 26 words "
  "at 1 star to 15 at 5. A whole-sample rank correlation had hidden this entirely.",
  "Fig 8", "High - the decline is monotone across all five levels"),
 ("Q4 Time",
  f"Volume is dominated by one collection spike ({monthly['reviews'].max()/len(df)*100:.0f}% of "
  "all reviews in a single month) while mean rating stays broadly flat.",
  "Fig 6", "Low for trend claims - the sampling is not uniform in time"),
 ("Q5 Recommendation",
  f"Tracks rating closely and monotonically ({rec.min():.0f}% at 1 star to {rec.max():.0f}% at 5), "
  "so it adds little beyond the rating - but it is missing for ~43% of rows.",
  "Fig 7", "Medium - restricted to the subset where the flag exists"),
 ("Q6 Structure",
  "Yes - three describable groups: brief praise (88%), critical detail (10%) and an engaged "
  "middle (2%) that writes long reviews and collects most of the helpful votes.",
  "Fig 10", "Medium - silhouette 0.55 at k=3, reasonable but not sharp"),
]
out = pd.DataFrame(ans, columns=["Question", "Answer", "Evidence", "Confidence"])
for _, r in out.iterrows():
    print(f"--- {r['Question']}  [{r['Evidence']}]")
    print(f"    {r['Answer']}")
    print(f"    confidence: {r['Confidence']}\n")
```

Output

```
--- Q1 Rating distribution  [Fig 3]
    Strongly left-skewed: mean 4.52, median 5, 90% are 4-5 stars.
    confidence: High - the whole dataset supports it

--- Q2 Category effects  [Fig 4, Fig 11]
    Yes, but small: mean rating spans 4.45 to 4.57 across categories with 200+ reviews.
    confidence: Medium - two categories dominate the volume

--- Q3 Text and rating  [Fig 8]
    Confirmed, and the effect is large: median length falls at every rating step, from 26 words at 1 star
to 15 at 5. A whole-sample rank correlation had hidden this entirely.
    confidence: High - the decline is monotone across all five levels

--- Q4 Time  [Fig 6]
    Volume is dominated by one collection spike (15% of all reviews in a single month) while mean rating
stays broadly flat.
    confidence: Low for trend claims - the sampling is not uniform in time

--- Q5 Recommendation  [Fig 7]
    Tracks rating closely and monotonically (3% at 1 star to 100% at 5), so it adds little beyond the
rating - but it is missing for ~43% of rows.
    confidence: Medium - restricted to the subset where the flag exists

--- Q6 Structure  [Fig 10]
    Yes - three describable groups: brief praise (88%), critical detail (10%) and an engaged middle (2%)
that writes long reviews and collects most of the helpful votes.
    confidence: Medium - silhouette 0.55 at k=3, reasonable but not sharp
```

Surprises, problems and what this data cannot answer

Three surprises

1. **A near-zero correlation concealed a large, consistent effect.** Spearman's rho between

rating and review length is about −0.06. Taken alone, that closes the question. Splitting by rating shows median length falling at *every* step — 26 words at 1 star down to 15 at 5, so dissatisfied customers write roughly 1.7 times as much as satisfied ones. The coefficient missed it because 86% of reviews sit at 4–5 stars and the within-group spread is far larger than the between-group shift. This is the strongest practical argument in the report for pairing every summary statistic with a plot.

1. **The most negative reviews are also the most read.** One-star reviews average far more

helpful votes than any other rating band. Readers seek out warnings. Combined with the length finding, this makes 1–2 star reviews the highest-value written feedback in the dataset — they are both the most detailed and the most influential.

1. **A small "engaged middle" carries the influence.** Roughly 2% of reviews form a group that

rates positively (4.5 average) but writes at length (median 48 words) and collects around 11 helpful votes each — far above the 0.04 of the silent majority. These are the reviews other customers actually read, and a natural target for a reviewer-incentive programme.

1. **The "would recommend" flag adds almost nothing.** It rises monotonically with rating and is

missing for around 43% of rows. A dashboard could drop it and lose very little.

Data quality problems encountered

Problem	Scale	How it was handled	Residual risk
`reviews.didPurchase` effectively empty	99.97% missing	Column dropped	None — but verified-purchase analysis is impossible
`doRecommend` / `numHelpful` missing	~43%	Analysed on the observed subset only	Missingness may not be random; results may not generalise
Duplicate reviews	removed in cleaning	Dropped on user + text + product	Some legitimate repeat reviews may have been lost
Volume concentrated in one month	one month holds a large share	Reported, not corrected	**Time trends are unreliable**
Extreme product concentration	top 5 products dominate	Reported alongside every category average	Category effects partly confound product effects

What this dataset cannot support

- **Causal claims.** Nothing here shows that a category *causes* higher ratings; product mix,

price and reviewer self-selection are all unobserved.

- **Trend claims.** The collection is not uniform in time, so any month-on-month movement may be

an artefact of when data was gathered.

- **Population claims.** These are reviewers who chose to write, on one retailer, for 65 products

from a handful of brands. They are not customers in general, and the sampling frame must be stated with any finding.

- **Verified-purchase claims.** The field exists but is empty.

If the work continued

Add price and units sold to test whether rating tracks value for money; apply topic modelling to the 2–3 star reviews to name the recurring trade-offs; and obtain a reviewer-level identifier to check whether a small number of prolific reviewers drive the averages.

Method summary and what we learned

Method, in the order it was carried out

1. **Framed the domain and wrote the questions first**, so the analysis was directed rather than

opportunistic. Three sanity checks were stated in advance; all four checks reported in the notebook passed except the uniform-time-coverage check, which failed informatively.

1. **Profiled before computing.** Shape, dtypes, cardinality and missingness were established

before any statistic was reported.

1. **Audited quality against six dimensions** — accuracy, completeness, consistency, validity,

uniqueness, timeliness — and recorded every finding.

1. **Cleaned with a log.** Five steps, each with the rows lost and the justification. Roughly 2%

of rows were removed.

1. **Converted types deliberately.** Rating became an *ordered* categorical, brands became

categories, dates were parsed, and five derived fields were constructed.

1. **Univariate, then bivariate, then multivariate** — in that order, non-graphical first and

graphical second, reconciling the two when they disagreed.

1. **Clustered exploratorily** and reported the weak silhouette honestly.
2. **Built the dashboard last**, from aggregations that refresh automatically.

What we learned about doing EDA

- **A summary statistic without a plot is an unverified claim.** The rho = −0.06 episode is the

clearest lesson in this report.

- **The denominator belongs beside every rate.** Category averages computed on 40 reviews were

greyed out in Figure 4 precisely so they would not be read as comparable to averages on 12,000.

- **Data quality findings are results, not chores.** The empty `didPurchase` column and the

single-month volume spike each changed what the report is allowed to claim.

- **Transform before you cluster.** The first clustering attempt, on raw helpful-vote counts with

a skew near 67, spent a whole cluster on six viral reviews. Log-transforming the skewed features lifted the silhouette from roughly 0.35 to 0.55 and produced groups of usable size. The algorithm was never wrong; the input was.

- **Negative and weak results are worth reporting.** The uninformative recommend flag and the

unreliable time trend are both stated plainly rather than dressed up.

- **Cleaning decisions must be written down as they are made.** Reconstructing them afterwards is

unreliable, and the log is what makes the analysis reproducible by someone else.